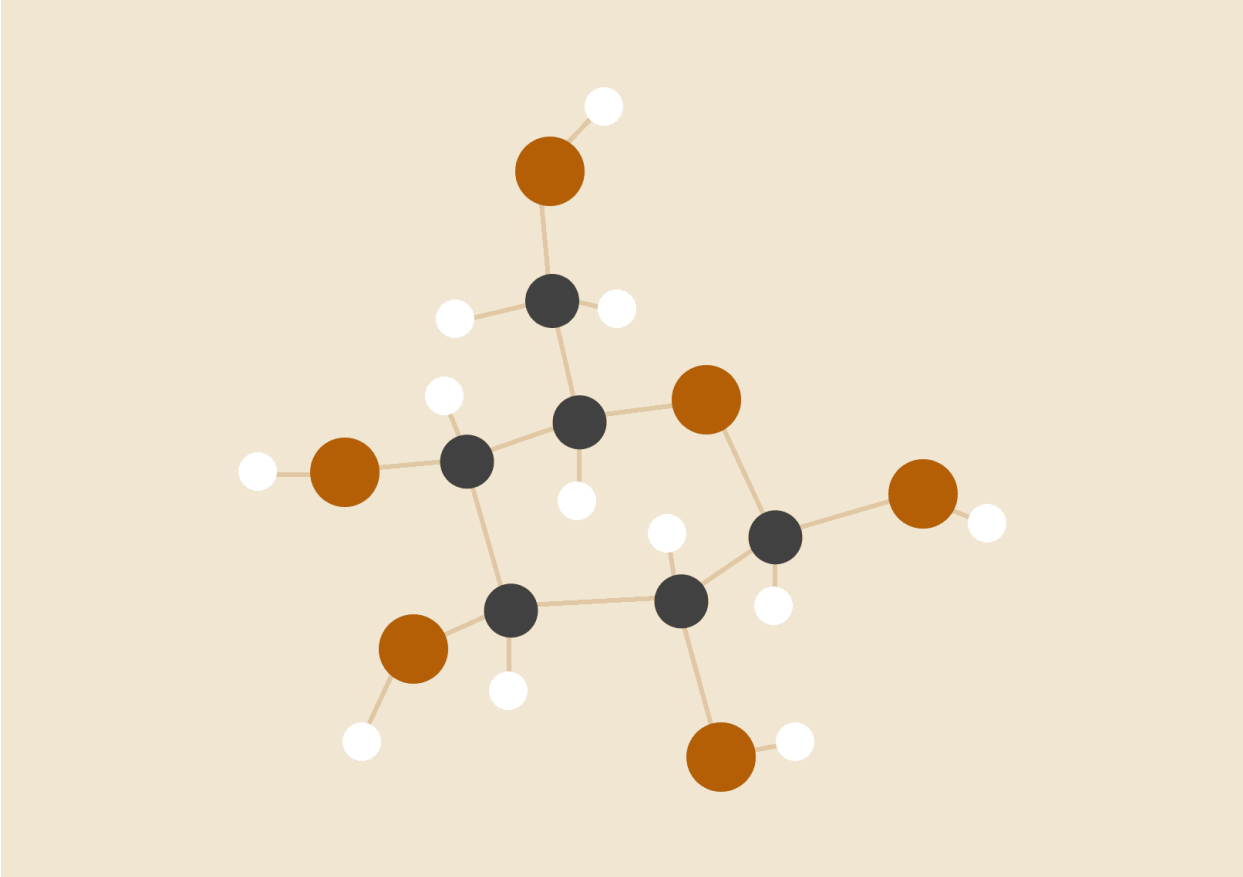
QUANTUM HVAC OPTIMIZATION



Raphael Langer

10.25.2026

Independent Project

PROBLEM AND CLASSICAL BASELINE

A building has three air conditioning (AC) units - A, B, and C. Their power metrics are as follows:

A consumes 10 kW and outputs 50 kW.

B consumes 17 kW and outputs 75 kW.

C consumes 25 kW and outputs 100 kW.

Each unit can be on or off at any given time, independent of the other units. Together, the units need to provide 100kW of cooling to the building. The problem is to find the optimal state that meets the cooling demand while minimizing the power consumption. On inspection, it's obvious that the optimal solution is A OFF, B OFF, C ON, as this produces exactly 100 kW of cooling with no excess power consumption. This report will investigate the ability of a quantum algorithm to independently solve the problem and find the optimal state.

QUBO FORMULATION

This problem can be formulated using a quadratic unconstrained binary objective (QUBO) cost function. Essentially, this formulation assigns a "cost" to each possible state, where the cost is the excess power consumption above the cooling demand plus a penalty if the cooling demand is not met. The general formula is:

$$F(\mathbf{z}) = K + \sum_{i=0}^{9} h_i z_i + \sum_{i<j} J_{ij} z_i z_j$$

Where F is the cost, z is the state expressed as binary variables, K is a constant, and h is the diagonal terms of a classical cost matrix (calculated separately), and J is the off-diagonal terms. Translating to an HVAC context, the QUBO can be written as:

$$F(\mathbf{z}) = 10a + 17b + 25c + \lambda \left(50a + 75b + 100c - 100 - \sum_{j=0}^{6} 2^j y_j \right)^2$$

Here, [a, b, c] are binary variable indicating the ON/OFF status of each unit, and y is a list of slack bits that encode the cooling overshoot above 100 kW. For example, if the overshoot is 20kW, we set y2 = 1 and y4 = 1, which gives 1 x 2^2 + 1 x 2^4 = 20. In our case, the maximum possible overshoot is whe all units are on, giving (50 + 75 + 100) - 100 = 125 kW. As a result, 7 slack bits (y_0 through y_6) are required to encode the overshoot, as 2^0 + … + 2^6 = 127 > 125. Thus, the z input variable is a vector of 10 binary variables, including 3 HVAC unit ON/OFF bits and 7 slack bits (in that order).

λ is the penalty factor that only kicks in if the cooling load is not met, or if a cooling-compliant configuration has slack-bits that so not correctly encode the cooling excess. If λ is high enough, the function will ensure that insufficient states are always more costly than states in excess of the cooling requirement.

ISING CONVERSION

The above QUBO can be converted into a quantum-optimizable form by replacing each binary variable in z with it's spin equivalent:

$$z_i = \frac{1 - s_i}{2}$$

Looking at the above formula, it is evident that z = 0 gives s = +1, and z = 1 gives s = -1. That transforms the QUBO into:

$$F(\mathbf{s}) = c + \sum_i \alpha_i s_i + \sum_{i<j} \beta_{ij} s_i s_j$$

Where α and β are coefficients that emerge after expanding the equation using the spin-variable substitutions. To make this equation compatible with a quantum circuit implementation, we can replace s_i with a Pauli-Z operator acting on qubit i (and similarly for s_j). As a reminder, the basic Z-operator is [[1, 0], [0, -1]], and a Z-operation on qubit i is a tensor product with the identity matrix. For example, the Z-operator for qubit 2 (=3rd qubit) is:

$$I \otimes I \otimes Z \otimes I \otimes I \otimes I \otimes I \otimes I \otimes I \otimes I$$

Making these substitutions converts the equation into a matrix:

$$H_C = cI + \sum_i \alpha_i Z_i + \sum_{i<j} \beta_{ij} Z_i Z_j$$

H_c is known as the Ising Hamiltonian, where each diagonal element is the QUBO cost of the corresponding state (assuming a cartesian state-ordering convention). For example, $H_c(2,2)$ is the QUBO cost of the 3rd ordered state.

QAOA CIRCUIT AND OPTIMIZATION

The Ising Hamiltonian can be converted into a series of quantum circuit gates by performing a matrix exponentiation, i.e.:

$$U_C(\gamma) = e^{-i\gamma H_C}$$

Expanding:

$$U_C(\gamma) = \exp\left[-i\gamma\left(cI + \sum_i \alpha_i Z_i + \sum_{i<j} \beta_{ij} Z_i Z_j\right)\right]$$

Where γ is a "variatonal angle" that can be tuned for better performance. U_c is known as the cost unitary, as it essentially applies a series of cost-based unitary matrices to each binary variable (i.e. bit) in z. The cost unitary does not need to be exponentiated directly; it can be written as a quantum circuit that uses a series of RZ and RZZ operators on each bit. These operators are defined as:

$$R_Z(\theta) = e^{-i\frac{\theta}{2}Z} = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$$

$$R_{ZZ}(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 & 0 & 0 \\ 0 & e^{i\theta/2} & 0 & 0 \\ 0 & 0 & e^{i\theta/2} & 0 \\ 0 & 0 & 0 & e^{-i\theta/2} \end{pmatrix}$$

Where $\theta = 2\gamma\alpha_i$ for R_Z, and $\theta = 2\gamma\beta_{ij}$ for R_{ZZ}. Applying these gates to each qubit essentially changes the phases of the state vector, which is a 2^{10} = 1,024-element vector representing the amplitudes of each possible basis state within the 10-bit system. The cost unitary can change the complex phase of each amplitude but will not alter the amplitude's absolute value.

Consequently, a mixer unitary should be applied to swap amplitudes between different basis states. This allows the circuit to explore different superpositions and possibly find the position that favors lower-cost states with higher amplitudes (i.e. higher probabilities of being measured). The mixer unitary can be written as:

$$U_M(\beta) = e^{-i\beta H_M}$$

Where β is the variational mixing angle, tuneable for performance. H_M is the Mixer Hamiltonian, defined as:

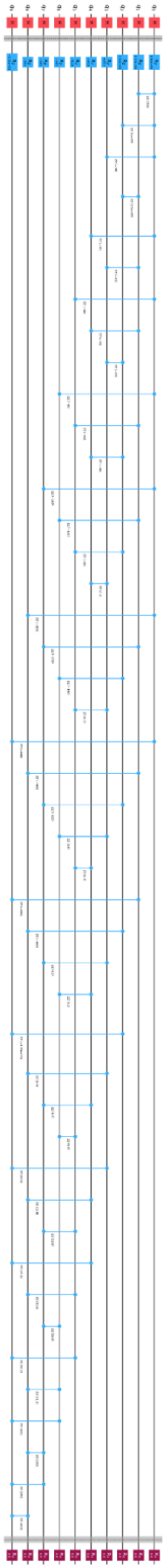
$$H_M = \sum_{i=0}^{n-1} X_i$$

X_i is the Pauli-X operator ([[0, 1], [1, 0]]) acting on qubit i. The mixer unitary can also be implemented as a quantum circuit, this time using the R_X gate:

$$R_X(\theta) = e^{-i\frac{\theta}{2}X} = \begin{pmatrix} \cos(\theta/2) & -i\sin(\theta/2) \\ -i\sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$$

With the definition $\theta = 2\beta$. Note that the β of mixing is not the related to the β coefficients from the Ising QUBO formulation.

If we use a Hadamard gate to initialize the 10-qubit state to the |+> state where all amplitudes are equal, and then implement the cost unitary followed by the mixer unitary, we get the following quantum circuit:



γ and β can now be tuned using a classical optimizer. The goal of tuning is to find values of γ and β that produce a circuit whose resulting state vector favors low-cost states with high probabilities.

IDEAL AND SHOT-BASED RESULTS

After running a classical optimizer using SciPy, the following optimal values for γ and β were found:

$$\gamma = 0.7046, \beta = -0.3453$$

Note that this assumes a one-layer quantum circuit (mutliple-layer circuits were explored but are not discussed in this report since the available hardware did not support the multiple-layer circuits). A circuit with these angles yields a state vector with amplitudes for each of 1,024 basis states; multiplying the QUBO cost of each basis state by the corresponding probability (i.e. amplitude squared) gives the expected cost:

```
Expected cost = 2372.575
Reduction in cost compared to uniform distribution =
1 - 2372.6 / 8523.5 =     72.2%
```

Note that the expected cost of a uniform distribution is the cost we would calculated if all basis-state amplitudes were equal, i.e. if the |+> state, where each amplitude is 1 / sqrt(1024). Significantly, this circuit was able to improve the expected cost by over 70% as compared to the initial uniform |+> state.

We can simulate this circuit using a Qiskit AerSimulator with 10,000 shots, where a “shot” means running the |+> initial state through the circuit and then converting the resulting state vector into 10 classical bit measurements. This is an “ideal” simulation in the sense that it assumes zero noise. Running the simulation gives:

```
Ideal expected cost = 2433.2
Ideal probability of optimal state (A OFF, B OFF, C ON, all slack bits OFF) =
0.25%
Ideal probability of feasible state (cooling >= 100 kW) = 84.0%
```

If we run the ideal circuit with different shot-counts, we can compare the effect of

increasing or decreasing the sample size on circuit:

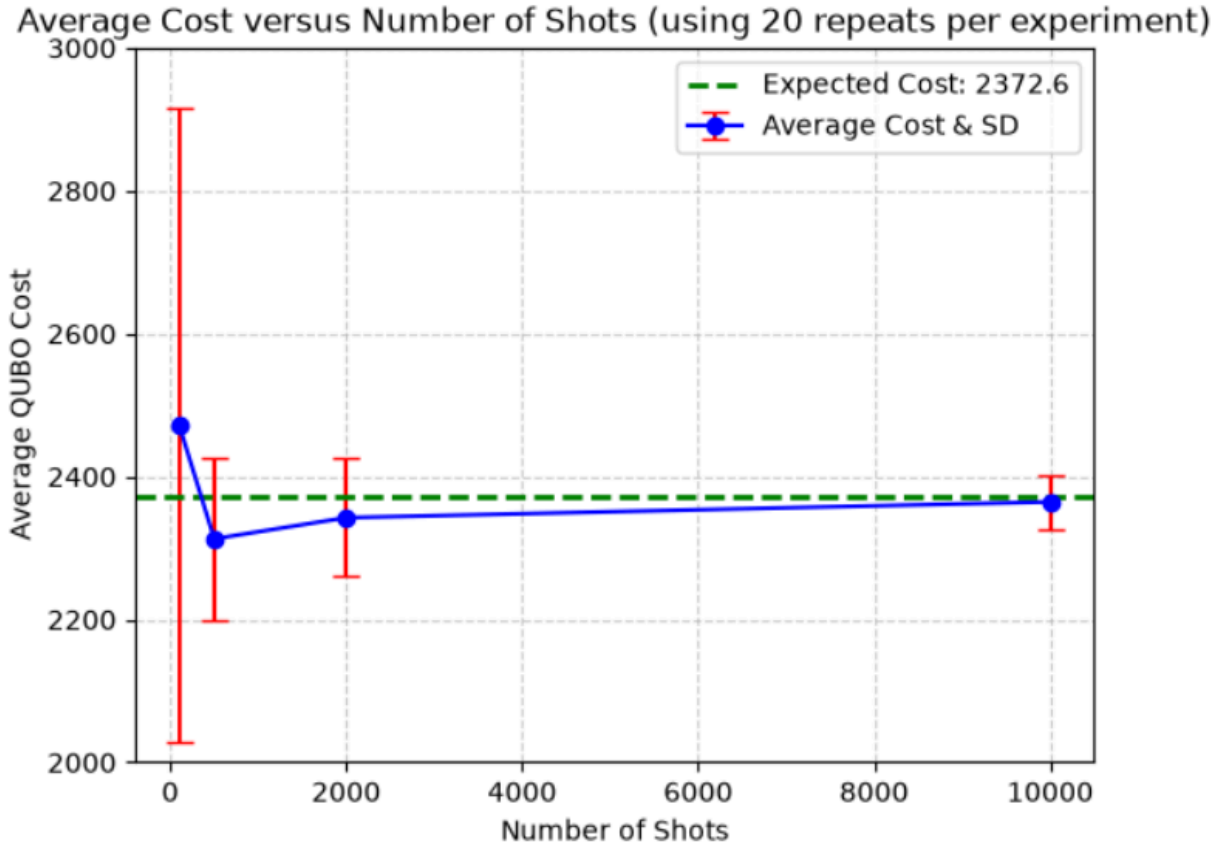


Figure 1. Average QUBO cost vs. shot-count.

Figure 1 shows that increasing shot-count generally leads to average costs that are closer to the exact one-layer expected cost. In other words, a higher shot count usually reduces sampling uncertainty. The results are also more consistent across repeated trials when using a higher shot-count, as indicated by the smaller standard deviations.

NOISE

If we introduce noise into the circuit and run the simulation again with the same number of shots, we get:

```
Depolarizing error  = 0.1% on single-qubit gates (rx, rz), 1.0% on two-qubit
                                        gates (rzz)
                    Measurement readout error = 2.0%
```

```
                    Noisy expected cost = 3125.4
  Noisy probability of optimal state (A OFF, B OFF, C ON, all slack bits OFF) =
                                    0.23%
Noisy probability of feasible state (cooling >= 100 kW) = 81.4%
```

Note that the depolarizing error introduces random Pauli errors into the quantum gate operations, and the readout error means that the simulator's may sometimes report an incorrect measurement. Interestingly, the noise increased the expected cost by a large margin but did not significantly change the optimal-state or feasible-state probabilities.

HARDWARE SETUP

We can run the one-layer circuit with the above optimized angles on IBM quantum processors. First the circuit needs to be transpiled so its gate sequence is compatible with the available IBM processor (IBM Kingston, job ID #dbf8b77pc3tc739muohg); transpilation gives the following data:

```
Original depth:  21

Transpiled depth:  250

Transpiled gate counts:

sx: 317

rz: 249

cz: 148

measure: 10

x: 4

barrier: 3

2-qubit gate count:  148
```

As is evident, the transpilation significantly increased the number of gates required to implement the circuit. This likely increases the overall hardware noise and lessens the circuit's cost-optimization performance.

Running the transpiled circuit on the IBM processor gives the following results:

```
Total shots:  2000

Number of distinct outcomes:  700

Hardware's average QUBO cost: 3977.1

Probability of optimal state (A OFF, B OFF, C ON, all slack bits OFF) using
hardware: 0.4%
Probability of measuring a feasible state (cooling >= 100 kW) using hardware:
70.4%
```

As suggested above, the circuit's hardware performance is not necessarily superior to its simulated performance - expected cost is higher and the feasible-state probability is lower, but the optimal-state probability is higher. These mixed effects are likely due to increased noise on the quantum hardware. Evidently, noise can redistribute probability toward some states while degrading the overall distribution.

COMPARISON OF RESULTS

Table 1 summarizes the results of the quantum HVAC optimizer circuit under the various conditions outline above.

	Method	Average Expected QUBO Cost	Probability of Optimal Outcome (%)	Probability of Feasible Outcome (%)	Shot Count	Circuit Depth
0	Classical Optimum	25.000000	100	100	N/A	N/A
1	Ideal Exact QAOA (1 layer)	2372.600000	0.20406%	84.47307%	N/A	21
2	Ideal Finite-Shot QAOA (1 layer)	2433.200000	0.250000	84.000000	10000	21
3	Noisy Simulator (1 layer)	3125.400000	0.230000	81.400000	10000	248
4	IBM Hardware (1 layer)	3977.100000	0.400000	70.400000	2000	250

Table 1. Results summary.

Clearly, the circuit's best performance was under noise-free simulated conditions.

However, the ideal simulation still fell far short of the expected cost for the true classical optimum. This is likely due to the large penalty factor generating significant cost-differences between feasible and non-feasible states; a single non-feasible state can skew the expected cost by a considerable margin. Additionally, the relatively poor hardware performance indicates that circuits of this formulation may be difficult to implement on hardware after transpilation; it is possible that alternative gate-types should be considered.

LIMITATIONS AND NEXT STEPS

This experiment demonstrates that a quantum circuit can, in principle, be used to optimize HVAC unit sequencing with adequate results under the right conditions. However, it does not demonstrate quantum advantage for HVAC applications at this scale. The biggest limitations of this approach are the skew produced by the penalty factor and the noise present in available quantum hardware. The next step in improving this model would be to formulate an objective with a more balanced penalty and develop a simplified circuit that transpiles more easily into a hardware-compatible version.